

F450 Drone PID Cascade Controller

Tuan Anh - dolphiinn

July 1, 2026

Contents

1	Kiến trúc tổng thể của bộ điều khiển F450	3
1.1	Luồng dữ liệu điều khiển	3
1.2	Vì sao dùng cấu trúc cascade	4
1.3	Chu kỳ điều khiển trong <code>on_physics_step</code>	4
1.4	Các tham số mặc định quan trọng	5
1.5	Các hàm tiện ích chung	5
2	Position Hold PID	6
2.1	Công thức PID chuẩn	6
2.2	Sai số vị trí trong Position Hold	6
2.3	Sai số vận tốc	7
2.4	Tích phân sai số	7
2.5	PID vị trí tạo gia tốc mong muốn	8
2.6	Bản chất vật lý của việc chọn gia tốc làm output	8
2.7	Giới hạn gia tốc ngang	9
2.8	Đổi từ [W] sang [B]	9
2.9	Bản chất vật lý - nghiêng drone để tạo gia tốc ngang	10
2.10	Tính pitch target	11
2.11	Tính roll target	11
2.12	Giới hạn góc nghiêng	12
2.13	Công thức tổng hợp của Position Hold PID	12
3	Cách xác định output của một bộ PID	14
3.1	Output của các tầng PID trong project	14
3.1.1	Position PID	14
3.1.2	Altitude PID	14
3.1.3	Attitude PID	15
3.1.4	Yaw PID	15
3.2	Mixer	16
3.3	Tóm tắt nguyên tắc chọn output PID	16
4	Altitude Hold PID	18
4.1	Sai số độ cao và vận tốc đứng	18
4.2	Tích phân độ cao	18
4.3	Output của Altitude PID	18
4.4	Ý nghĩa vật lý	19

5	Attitude Hold PID	19
5.1	Sai số roll/pitch	19
5.2	Thành phần vi phân dùng vận tốc góc	19
5.3	Tích phân và giới hạn output	20
6	Yaw Hold PID	20
6.1	Sai số yaw	20
6.2	Công thức yaw correction	21
6.3	Vai trò của yaw correction trong quadcopter	21
7	Quad X PWM Mixer	22
7.1	Công thức mixer	22
7.2	Ý nghĩa dấu trong mixer	22
8	Motor Model	23
8.1	Đổi PWM sang throttle	23
8.2	Dòng điện và torque motor	23
8.3	Động lực học rotor	23
8.4	Lực đẩy, torque phản lực và RPM	24
9	Apply lực vào Isaac Sim	24
9.1	Torque phản lực yaw	25
9.2	Kết nối giữa motor model và simulator	25
10	Propeller Spinner	26
10.1	Đổi RPM sang vận tốc joint	26
11	Tracking Logger	26
11.1	Các đại lượng được ghi	26
12	Compatibility Mixin	27
13	Tóm tắt toàn bộ hệ thống	27

1 Kiến trúc tổng thể của bộ điều khiển F450

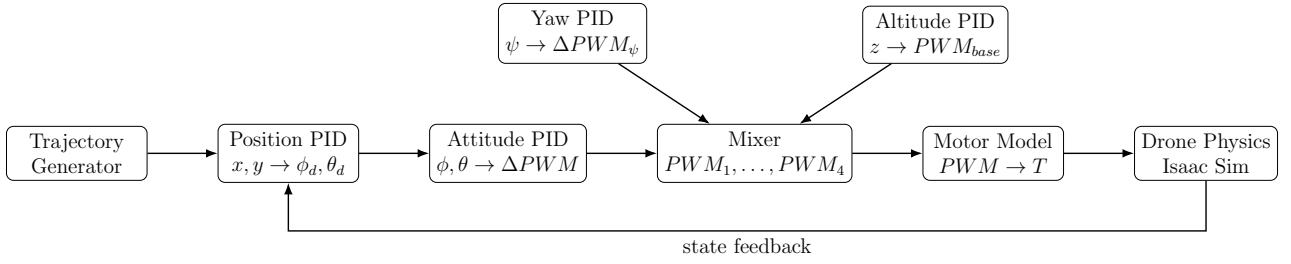


Figure 1: Cascade PID control structure

Phần điều khiển chính của drone F450 được tổ chức theo dạng *cascade PID controller*. Lớp trung tâm là `F450AttitudeHold` trong file `issac_attitude_hold.py`. Lớp này không chứa toàn bộ thuật toán trong một khối lớn, mà ghép nhiều module nhỏ lại với nhau:

- `PositionHoldPID`: giữ vị trí ngang x, y và tạo $roll_target, pitch_target$.
- `AltitudeHoldPID`: giữ độ cao z và tạo PWM_{base} .
- `AttitudeHoldPID`: giữ roll/pitch và tạo $\Delta PWM_{\phi}, \Delta PWM_{\theta}$.
- `YawHoldPID`: giữ yaw và tạo ΔPWM_{ψ} .
- `QuadXPwmMixer`: trộn PWM base với các PWM hiệu chỉnh - ΔPWM để tạo bốn lệnh motor.
- `MotorModel`: đổi PWM thành lực đẩy, dòng điện, RPM và torque phản lực của từng motor.
- `PhysicalPropellerSpinner`: làm quay joint cánh quạt vật lý theo RPM mô phỏng.
- `TrackingLogger`: ghi trạng thái và target ra file CSV để phân tích sau khi bay.

1.1 Luồng dữ liệu điều khiển

Trong mỗi bước vật lý của Isaac Sim, controller đọc trạng thái hiện tại của drone:

$$x, y, z, \quad v_x, v_y, v_z, \quad \phi, \theta, \psi, \quad p, q, r \quad (1)$$

Trong đó ϕ là roll, θ là pitch, ψ là yaw, còn p, q, r là vận tốc góc quanh các trục body. Sau đó luồng điều khiển chính là:

$$x_d, y_d, x, y, v_x, v_y, \psi \longrightarrow PositionPID \longrightarrow \phi_d, \theta_d \quad (2)$$

$$z_d, z, v_z \longrightarrow AltitudePID \longrightarrow PWM_{base} \quad (3)$$

$$\phi_d, \theta_d, \phi, \theta, p, q \longrightarrow AttitudePID \longrightarrow \Delta PWM_{\phi}, \Delta PWM_{\theta} \quad (4)$$

$$\psi_d, \psi, r \longrightarrow YawPID \longrightarrow \Delta PWM_{\psi} \quad (5)$$

Các output trên được đưa vào mixer:

$$PWM_{base}, \Delta PWM_{\phi}, \Delta PWM_{\theta}, \Delta PWM_{\psi} \longrightarrow Mixer \longrightarrow PWM_1, PWM_2, PWM_3, PWM_4 \quad (6)$$

Cuối cùng, từng lệnh PWM đi qua motor model để tạo lực đẩy:

$$PWM_i \longrightarrow MotorModel_i \longrightarrow T_i, RPM_i, Q_i \quad (7)$$

Trong đó T_i là lực đẩy của motor thứ i , còn Q_i là torque phản lực do cánh quạt tạo ra.

1.2 Vì sao dùng cấu trúc cascade

Drone là hệ động lực học nhiều tầng. Vị trí không thể điều khiển trực tiếp bằng PWM, vì PWM trước hết tạo lực đẩy, lực đẩy tạo moment và gia tốc, rồi gia tốc mới làm thay đổi vị trí. Vì vậy project chia bài toán thành nhiều tầng:

- Tầng ngoài cùng - outer loop quyết định drone cần nghiêng bao nhiêu để quay về vị trí mong muốn.
- Tầng độ cao quyết định tổng lực nâng cần tăng hay giảm bao nhiêu.
- Tầng attitude làm drone bám theo góc roll/pitch mong muốn.
- Tầng yaw giữ hướng quay quanh trục đứng.
- Mixer biến các lệnh trừu tượng thành lệnh cụ thể cho bốn motor.

Cách chia tầng này giúp mỗi PID có một nhiệm vụ rõ ràng. Position PID không cần biết chi tiết từng motor; attitude PID không cần biết target vị trí đến từ đâu; motor model chỉ cần nhận PWM và trả về lực.

1.3 Chu kỳ điều khiển trong on_physics_step

Hàm `on_physics_step(dt)` là vòng lặp điều khiển được gọi mỗi bước physics. Các bước chính là:

1. Lấy rigid body `base_link` từ `dynamic_control`.
2. Đọc pose, vận tốc tuyến tính và vận tốc góc.
3. Đổi quaternion sang Euler bằng `quat_to_euler`.
4. Nếu chưa đặt target x, y , tự lấy vị trí ban đầu làm target giữ vị trí.
5. Tính target roll/pitch từ position controller.
6. Tính PWM_{base} từ altitude controller.
7. Tính yaw correction nếu yaw hold đang bật.
8. Tính roll/pitch correction từ attitude controller.
9. Trộn bốn lệnh PWM bằng mixer.
10. Cập nhật bốn motor model và apply lực đẩy vào Isaac Sim.
11. Tính torque phản lực yaw, quay cánh quạt vật lý, ghi log và in debug.

Như vậy, Isaac Sim vừa là môi trường vật lý nhận lực từ controller, vừa là cảm biến trả lại trạng thái cho controller ở bước sau.

1.4 Các tham số mặc định quan trọng

Nhóm	Tham số	Giá trị mặc định trong code
PWM	PWM_{min}, PWM_{max}	1100, 1940
F450 main	z_d	1.5 m
F450 main	PWM_{hover}	1650
Position	$K_{p_x}, K_{d_x}, K_{i_x}$	0.55, 1.30, 0.03
Position	$K_{p_y}, K_{d_y}, K_{i_y}$	0.55, 1.30, 0.03
Position	giới hạn gia tốc	1.20 m/s ²
Position	giới hạn góc	6.5°
Altitude	$K_{p_z}, K_{d_z}, K_{i_z}$	220, 150, 15
Attitude	K_p, K_i, K_d roll/pitch	520, 25, 130
Yaw	$K_{p_\psi}, K_{i_\psi}, K_{d_\psi}$	95, 8, 28

Lưu ý: các giá trị này có thể thay đổi trong lúc chạy bằng cách áp đặt giá trị mới vào Script Editor của Isaac Sim.

1.5 Các hàm tiện ích chung

File `control_utils.py` chứa các hàm nhỏ nhưng quan trọng:

$$\text{clamp}(x, l, h) = \max(l, \min(x, h)) \quad (8)$$

Hàm này dùng để giới hạn PWM, góc, gia tốc và tích phân PID trong vùng an toàn.

Hàm `wrap_angle` đưa góc về khoảng:

$$-\pi \leq \text{angle} \leq \pi \quad (9)$$

Nhờ đó sai số góc luôn lấy theo đường ngắn nhất. Ví dụ, sai số giữa 179° và −179° phải là 2°, không phải 358°.

Hàm `quat_to_euler` đổi quaternion của Isaac Sim sang roll, pitch, yaw. Đây là bước cần thiết vì controller attitude và yaw làm việc trực tiếp với góc Euler.

2 Position Hold PID



Figure 2: Position PID control law

2.1 Công thức PID chuẩn

Công thức PID tổng quát có dạng [1]:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \dot{e}(t) \quad (10)$$

trong đó:

$$e(t) = r(t) - y(t) \quad (11)$$

Với:

- $r(t)$: giá trị mong muốn,
- $y(t)$: giá trị đo được,
- $e(t)$: sai số,
- $u(t)$: output điều khiển,
- K_p, K_i, K_d : hệ số tỉ lệ, tích phân và vi phân.

Trong bộ điều khiển drone, output $u(t)$ không nhất thiết là PWM. Tùy tầng điều khiển, $u(t)$ có thể là gia tốc, góc mong muốn, moment, lực nâng hoặc PWM.

2.2 Sai số vị trí trong Position Hold

Gọi vị trí mong muốn của drone trong WORLD-coordination [W] là:

$$x_d, \quad y_d \quad (12)$$

Vị trí hiện tại của drone là:

$$x, \quad y \quad (13)$$

Sai số vị trí được định nghĩa là:

$$e_x = x_d - x \quad (14)$$

$$e_y = y_d - y \quad (15)$$

Nếu $e_x > 0$, drone đang nằm phía sau target theo trục x và cần tạo gia tốc theo chiều $+x$.
Nếu $e_y > 0$, drone đang lệch so với target theo trục y và cần tạo gia tốc theo chiều $+y$.

2.3 Sai số vận tốc

Gọi vận tốc mong muốn là:

$$v_{xd}, \quad v_{yd} \quad (16)$$

Vận tốc hiện tại là:

$$v_x, \quad v_y \quad (17)$$

Sai số vận tốc là:

$$e_{v_x} = v_{xd} - v_x \quad (18)$$

$$e_{v_y} = v_{yd} - v_y \quad (19)$$

Ta có:

$$e_x = x_d - x \quad (20)$$

Lấy đạo hàm hai vế theo thời gian:

$$\dot{e}_x = \dot{x}_d - \dot{x} \quad (21)$$

Mà:

$$\dot{x}_d = v_{xd} \quad (22)$$

$$\dot{x} = v_x \quad (23)$$

Do đó:

$$\dot{e}_x = v_{xd} - v_x = e_{v_x} \quad (24)$$

Tương tự:

$$\dot{e}_y = v_{yd} - v_y = e_{v_y} \quad (25)$$

Vì vậy, thành phần sai số vận tốc e_{v_x} và e_{v_y} chính là thành phần vi phân của sai số vị trí.

2.4 Tích phân sai số

Thành phần tích phân của sai số vị trí là:

$$I_x = \int e_x dt \quad (26)$$

$$I_y = \int e_y dt \quad (27)$$

Trong dạng rời rạc:

$$I_x(k) = I_x(k-1) + e_x(k)\Delta t \quad (28)$$

$$I_y(k) = I_y(k-1) + e_y(k)\Delta t \quad (29)$$

Để tránh hiện tượng tích phân tăng quá lớn, ta giới hạn:

$$-I_{\max} \leq I_x \leq I_{\max} \quad (30)$$

$$-I_{\max} \leq I_y \leq I_{\max} \quad (31)$$

Giới hạn này giúp chống hiện tượng *integral windup*.

2.5 PID vị trí tạo gia tốc mong muốn

Trong tầng Position Hold, PID không tạo PWM trực tiếp. Output của PID vị trí là gia tốc ngang mong muốn:

$$a_{x,cmd} \quad (32)$$

$$a_{y,cmd} \quad (33)$$

Công thức PID theo trục x :

$$a_{x,cmd} = K_{p_x} e_x + K_{d_x} e_{v_x} + K_{i_x} I_x \quad (34)$$

Công thức PID theo trục y :

$$a_{y,cmd} = K_{p_y} e_y + K_{d_y} e_{v_y} + K_{i_y} I_y \quad (35)$$

So với công thức PID chuẩn:

$$u = K_p e + K_i \int e dt + K_d \dot{e} \quad (36)$$

thì trong tầng Position Hold:

$$u_x = a_{x,cmd} \quad (37)$$

$$u_y = a_{y,cmd} \quad (38)$$

Tức là output của PID vị trí là gia tốc ngang mong muốn, không phải PWM.

2.6 Bản chất vật lý của việc chọn gia tốc làm output

Theo định luật II Newton:

$$F = ma \quad (39)$$

Muốn thay đổi vị trí ngang của drone, ta cần tạo gia tốc ngang:

$$a_x, \quad a_y \quad (40)$$

Do đó, tầng Position Hold trả lời câu hỏi:

Drone cần gia tốc ngang bao nhiêu để quay về vị trí mong muốn?

Vì vậy, output hợp lý của tầng Position Hold là:

$$a_{x,cmd}, \quad a_{y,cmd} \quad (41)$$

Sau đó, các gia tốc này sẽ được chuyển thành góc nghiêng mong muốn.

2.7 Giới hạn gia tốc ngang

Vector gia tốc ngang mong muốn là:

$$\mathbf{a}_{cmd} = \begin{bmatrix} a_{x,cmd} \\ a_{y,cmd} \end{bmatrix} \quad (42)$$

Độ lớn vector gia tốc là:

$$\|\mathbf{a}_{cmd}\| = \sqrt{a_{x,cmd}^2 + a_{y,cmd}^2} \quad (43)$$

Project giới hạn gia tốc ngang theo điều kiện:

$$\|\mathbf{a}_{cmd}\| \leq a_{\max} \quad (44)$$

Nếu:

$$\sqrt{a_{x,cmd}^2 + a_{y,cmd}^2} > a_{\max} \quad (45)$$

thì scale lại:

$$a_{x,cmd}^{new} = a_{x,cmd} \frac{a_{\max}}{\sqrt{a_{x,cmd}^2 + a_{y,cmd}^2}} \quad (46)$$

$$a_{y,cmd}^{new} = a_{y,cmd} \frac{a_{\max}}{\sqrt{a_{x,cmd}^2 + a_{y,cmd}^2}} \quad (47)$$

Bản chất của bước này là giới hạn độ mạnh của lệnh điều khiển vị trí. Nếu gia tốc ngang quá lớn, drone phải nghiêng quá nhiều, dễ gây mất ổn định.

2.8 Đổi từ [W] sang [B]

Gia tốc $a_{x,cmd}$ và $a_{y,cmd}$ đang nằm trong [W]. Tuy nhiên, roll và pitch lại là góc nghiêng theo thân drone.

Gọi yaw hiện tại của drone là:

$$\psi \quad (48)$$

Vector gia tốc trong [W] là:

$$\mathbf{a}^W = \begin{bmatrix} a_x \\ a_y \end{bmatrix} \quad (49)$$

Trục forward của drone trong [W] là:

$$\mathbf{e}_f = \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} \quad (50)$$

Trục left của drone trong [W] là:

$$\mathbf{e}_l = \begin{bmatrix} -\sin \psi \\ \cos \psi \end{bmatrix} \quad (51)$$

Gia tốc theo hướng forward của drone là hình chiếu của vector gia tốc world lên trục forward:

$$a_f = \mathbf{a}^W \cdot \mathbf{e}_f \quad (52)$$

Suy ra:

$$a_f = a_x \cos \psi + a_y \sin \psi \quad (53)$$

Gia tốc theo hướng left của drone là:

$$a_l = \mathbf{a}^W \cdot \mathbf{e}_l \quad (54)$$

Suy ra:

$$a_l = -a_x \sin \psi + a_y \cos \psi \quad (55)$$

Trong đó:

- a_f : gia tốc mong muốn theo hướng tiến/lùi của drone,
- a_l : gia tốc mong muốn theo hướng trái/phải của drone.

2.9 Bản chất vật lý - nghiêng drone để tạo gia tốc ngang

Quadcopter tạo lực nâng tổng T theo trục thân của nó [2].

Khi drone nằm ngang, lực nâng hướng thẳng lên. Ở trạng thái hover:

$$T \approx mg \quad (56)$$

Khi drone nghiêng một góc α , lực nâng bị nghiêng theo. Khi đó lực nâng có thành phần đứng và thành phần ngang:

$$T_z = T \cos \alpha \quad (57)$$

$$T_h = T \sin \alpha \quad (58)$$

Để giữ độ cao gần như không đổi:

$$T \cos \alpha \approx mg \quad (59)$$

Theo phương ngang:

$$T \sin \alpha = ma_h \quad (60)$$

Chia hai phương trình:

$$\frac{T \sin \alpha}{T \cos \alpha} = \frac{ma_h}{mg} \quad (61)$$

Suy ra:

$$\tan \alpha = \frac{a_h}{g} \quad (62)$$

Do đó:

$$\alpha = \arctan \left(\frac{a_h}{g} \right) \quad (63)$$

Đây là công thức vật lý cơ bản để chuyển từ gia tốc ngang mong muốn sang góc nghiêng mong muốn.

2.10 Tính pitch target

Gọi gia tốc mong muốn theo hướng forward là:

$$a_f \quad (64)$$

Góc pitch mong muốn là:

$$\theta_d \quad (65)$$

Theo quan hệ vật lý:

$$\tan \theta_d = \frac{a_f}{g} \quad (66)$$

Suy ra:

$$\theta_d = \arctan \left(\frac{a_f}{g} \right) \quad (67)$$

Trong project, công thức được viết tương đương dưới dạng:

$$\theta_d = \text{atan2}(a_f, g) \quad (68)$$

Với góc nhỏ:

$$\theta_d \approx \frac{a_f}{g} \quad (69)$$

Trong đó:

$$\theta_d = \text{pitch_target} \quad (70)$$

2.11 Tính roll target

Gọi gia tốc mong muốn theo hướng left là:

$$a_l \quad (71)$$

Góc roll mong muốn là:

$$\phi_d \quad (72)$$

Theo quy ước dấu trong project:

$$\phi_d = -\text{atan2}(a_l, g) \quad (73)$$

Tương đương:

$$\phi_d = -\arctan \left(\frac{a_l}{g} \right) \quad (74)$$

Với góc nhỏ:

$$\phi_d \approx -\frac{a_l}{g} \quad (75)$$

Trong đó:

$$\phi_d = \text{roll_target} \quad (76)$$

Dấu âm xuất hiện do quy ước trục roll trong mô hình. Nếu sau này drone bay ngược hướng trái/phải, đây là một dấu cần kiểm tra.

2.12 Giới hạn góc nghiêng

Sau khi tính được ϕ_d và θ_d , project giới hạn:

$$-\alpha_{\max} \leq \phi_d \leq \alpha_{\max} \quad (77)$$

$$-\alpha_{\max} \leq \theta_d \leq \alpha_{\max} \quad (78)$$

Nếu:

$$\alpha_{\max} = 6.5^\circ \quad (79)$$

thì gia tốc ngang tối đa tương ứng xấp xỉ là:

$$a_{\max} = g \tan(\alpha_{\max}) \quad (80)$$

Thay số:

$$a_{\max} = 9.81 \tan(6.5^\circ) \quad (81)$$

$$a_{\max} \approx 1.12 \text{ m/s}^2 \quad (82)$$

Điều này cho thấy giới hạn góc nghiêng và giới hạn gia tốc ngang có liên hệ trực tiếp với nhau.

2.13 Công thức tổng hợp của Position Hold PID

Toàn bộ tầng Position Hold có thể viết tóm tắt như sau.

Sai số vị trí:

$$e_x = x_d - x \quad (83)$$

$$e_y = y_d - y \quad (84)$$

Sai số vận tốc:

$$e_{v_x} = v_{xd} - v_x \quad (85)$$

$$e_{v_y} = v_{yd} - v_y \quad (86)$$

Tích phân sai số:

$$I_x = \int e_x dt \quad (87)$$

$$I_y = \int e_y dt \quad (88)$$

PID tạo gia tốc ngang mong muốn:

$$a_x = K_{p_x} e_x + K_{d_x} e_{v_x} + K_{i_x} I_x \quad (89)$$

$$a_y = K_{p_y} e_y + K_{d_y} e_{v_y} + K_{i_y} I_y \quad (90)$$

Đổi sang hệ trục của drone:

$$a_f = a_x \cos \psi + a_y \sin \psi \quad (91)$$

$$a_l = -a_x \sin \psi + a_y \cos \psi \quad (92)$$

Đổi gia tốc thành góc nghiêng:

$$\theta_d = \text{atan2}(a_f, g) \quad (93)$$

$$\phi_d = -\text{atan2}(a_l, g) \quad (94)$$

3 Cách xác định output của một bộ PID

Một bộ PID không tự quyết định output là gia tốc, lực, moment hay PWM. Người thiết kế quyết định output dựa trên câu hỏi: "Tầng sau của PID cần nhận đại lượng gì?"

Có thể viết nguyên tắc này dưới dạng:

$$\boxed{\text{Đơn vị output của PID} = \text{Đơn vị input mà tầng sau cần}} \quad (95)$$

Ví dụ, nếu tầng sau cần gia tốc:

$$u = a \quad (96)$$

Nếu tầng sau cần lực:

$$u = F \quad (97)$$

Nếu tầng sau cần moment:

$$u = \tau \quad (98)$$

Nếu tầng sau cần PWM:

$$u = PWM \quad (99)$$

3.1 Output của các tầng PID trong project

3.1.1 Position PID

Tầng Position PID nhận sai số vị trí và biến nó thành lệnh gia tốc/góc nghiêng mong muốn [3]:

$$e_x, \quad e_y \quad (100)$$

và tạo gia tốc ngang mong muốn:

$$a_x, \quad a_y \quad (101)$$

Sau đó đổi thành góc nghiêng mong muốn:

$$a_x, \quad a_y \longrightarrow \phi_d, \quad \theta_d \quad (102)$$

Vì vậy:

$$\boxed{PositionPID \longrightarrow roll_target, \quad pitch_target} \quad (103)$$

3.1.2 Altitude PID

Tầng Altitude PID điều khiển độ cao z .

Sai số độ cao:

$$e_z = z_d - z \quad (104)$$

Sai số vận tốc đứng:

$$e_{v_z} = v_{zd} - v_z \quad (105)$$

Trong project, output của tầng này là PWM nên:

$$PWM_{base} = PWM_{hover} + K_{p_z}e_z + K_{d_z}e_{v_z} + K_{i_z} \int e_z dt \quad (106)$$

Vì vậy:

$$\boxed{AltitudePID \longrightarrow PWM_{base}} \quad (107)$$

3.1.3 Attitude PID

Tầng Attitude PID nhận góc mong muốn:

$$\phi_d, \quad \theta_d \quad (108)$$

và góc hiện tại:

$$\phi, \quad \theta \quad (109)$$

Sai số roll:

$$e_\phi = \phi_d - \phi \quad (110)$$

Sai số pitch:

$$e_\theta = \theta_d - \theta \quad (111)$$

Trong project, output của tầng này là PWM correction:

$$\Delta PWM_\phi = K_{p_\phi}e_\phi + K_{i_\phi} \int e_\phi dt - K_{d_\phi}p \quad (112)$$

$$\Delta PWM_\theta = K_{p_\theta}e_\theta + K_{i_\theta} \int e_\theta dt - K_{d_\theta}q \quad (113)$$

Trong đó:

$$p = \dot{\phi} \quad (114)$$

$$q = \dot{\theta} \quad (115)$$

Vì vậy:

$$\boxed{AttitudePID \longrightarrow \Delta PWM_\phi, \Delta PWM_\theta} \quad (116)$$

3.1.4 Yaw PID

Sai số yaw:

$$e_\psi = \text{wrap}(\psi_d - \psi) \quad (117)$$

Output của yaw PID là PWM correction theo yaw:

$$\Delta PWM_\psi = K_{p_\psi}e_\psi + K_{i_\psi} \int e_\psi dt - K_{d_\psi}r \quad (118)$$

Trong đó:

$$r = \dot{\psi} \quad (119)$$

Vì vậy:

$$\boxed{YawPID \longrightarrow \Delta PWM_{\psi}} \quad (120)$$

3.2 Mixer

Mixer nhận các đại lượng:

$$PWM_{base}, \quad \Delta PWM_{\phi}, \quad \Delta PWM_{\theta}, \quad \Delta PWM_{\psi} \quad (121)$$

và tạo PWM cho từng motor:

$$PWM_1 = PWM_{base} + \Delta PWM_{\phi} - \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (122)$$

$$PWM_2 = PWM_{base} - \Delta PWM_{\phi} - \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (123)$$

$$PWM_3 = PWM_{base} - \Delta PWM_{\phi} + \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (124)$$

$$PWM_4 = PWM_{base} + \Delta PWM_{\phi} + \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (125)$$

Vì vậy:

$$\boxed{Mixer \longrightarrow PWM_1, PWM_2, PWM_3, PWM_4} \quad (126)$$

3.3 Tóm tắt nguyên tắc chọn output PID

Nguyên tắc chung:

$$\boxed{\text{PID output phụ thuộc vào tầng sau cần gì}} \quad (127)$$

Trong project hiện tại:

$$PositionPID \longrightarrow a_x, a_y \longrightarrow \phi_d, \theta_d \quad (128)$$

$$AltitudePID \longrightarrow PWM_{base} \quad (129)$$

$$AttitudePID \longrightarrow \Delta PWM_{\phi}, \Delta PWM_{\theta} \quad (130)$$

$$YawPID \longrightarrow \Delta PWM_{\psi} \quad (131)$$

$$Mixer \longrightarrow PWM_1, PWM_2, PWM_3, PWM_4 \quad (132)$$

$$MotorModel \longrightarrow T_1, T_2, T_3, T_4 \quad (133)$$

Do đó, trong project này:

- Position PID tạo gia tốc rồi đổi thành góc nghiêng mong muốn.

- Altitude PID tạo PWM nền.
- Attitude PID tạo PWM hiệu chỉnh roll/pitch.
- Yaw PID tạo PWM hiệu chỉnh yaw.
- Mixer tạo PWM cho từng motor.
- Motor model biến PWM thành lực nâng.

4 Altitude Hold PID

Tầng altitude điều khiển độ cao z của drone. Khác với Position PID, tầng này không tạo góc nghiêng mà tạo PWM base cho cả bốn motor.

4.1 Sai số độ cao và vận tốc đứng

Gọi độ cao mong muốn là:

$$z_d \quad (134)$$

Độ cao hiện tại là:

$$z \quad (135)$$

Sai số độ cao:

$$e_z = z_d - z \quad (136)$$

Gọi vận tốc đứng mong muốn là v_{zd} và vận tốc đứng hiện tại là v_z . Sai số vận tốc đứng:

$$e_{v_z} = v_{zd} - v_z \quad (137)$$

Trong project, v_{zd} mặc định bằng 0, nghĩa là khi giữ độ cao thì drone không mong muốn tiếp tục đi lên hoặc đi xuống.

4.2 Tích phân độ cao

Thành phần tích phân của sai số độ cao:

$$I_z(k) = I_z(k-1) + e_z(k)\Delta t \quad (138)$$

Project giới hạn:

$$-I_{z,max} \leq I_z \leq I_{z,max} \quad (139)$$

Trong code:

$$I_{z,max} = 12.0 \quad (140)$$

Giới hạn này giúp tránh integral windup khi drone ở xa target hoặc khi PWM đã bị kẹp ở biên.

4.3 Output của Altitude PID

Altitude PID tạo ra lượng thay đổi PWM:

$$\Delta PWM_z = K_{p_z} e_z + K_{d_z} e_{v_z} + K_{i_z} I_z \quad (141)$$

Base PWM được tính bằng:

$$PWM_{base} = PWM_{hover} + \Delta PWM_z \quad (142)$$

Sau đó được giới hạn trong khoảng:

$$PWM_{min} \leq PWM_{base} \leq PWM_{max} \quad (143)$$

Với:

$$PWM_{min} = 1100, \quad PWM_{max} = 1940 \quad (144)$$

Do đó công thức đầy đủ trong code là:

$$PWM_{base} = clamp(PWM_{hover} + K_{p_z}e_z + K_{d_z}e_{v_z} + K_{i_z}I_z, 1100, 1940) \quad (145)$$

4.4 Ý nghĩa vật lý

PWM_{base} là mức ga chung cho bốn motor. Nếu drone thấp hơn target thì $e_z > 0$, khi đó ΔPWM_z tăng và tổng lực nâng tăng. Nếu drone cao hơn target thì $e_z < 0$, PWM nền giảm để drone hạ xuống.

Tầng altitude chỉ lo tổng lực nâng, còn việc phân bố lực giữa các motor để giữ roll, pitch, yaw sẽ do mixer xử lý.

5 Attitude Hold PID

Tầng attitude giữ roll và pitch của drone. Input của tầng này là góc mong muốn từ position controller:

$$\phi_d, \quad \theta_d \quad (146)$$

và góc hiện tại:

$$\phi, \quad \theta \quad (147)$$

Trong đó ϕ là roll và θ là pitch.

5.1 Sai số roll/pitch

Sai số roll:

$$e_\phi = wrap(\phi_d - \phi) \quad (148)$$

Sai số pitch:

$$e_\theta = wrap(\theta_d - \theta) \quad (149)$$

Hàm *wrap* giữ sai số góc trong khoảng $[-\pi, \pi]$.

5.2 Thành phần vi phân dùng vận tốc góc

Trong code, thành phần D của attitude PID không tính bằng:

$$\frac{de_\phi}{dt}, \quad \frac{de_\theta}{dt} \quad (150)$$

mà dùng trực tiếp vận tốc góc body:

$$p = \dot{\phi}, \quad q = \dot{\theta} \quad (151)$$

Nếu dùng PID chuẩn, thành phần D là:

$$K_{d\phi} \frac{de_\phi}{dt} = K_{d\phi} \left(\frac{d\phi_d}{dt} - \frac{d\phi}{dt} \right) = K_{d\phi} (\dot{\phi}_d - \dot{\phi}) \quad (152)$$

Nếu roll target thay đổi chậm: $\dot{\phi}_d \approx 0$ và $p \approx \dot{\phi}$
 Vì vậy:

$$K_{d\phi} \frac{de_\phi}{dt} \approx -K_{d\phi} \dot{\phi}_d = -K_{d\phi} p \quad (153)$$

Tương tự với pitch, đó là lí do công thức có dấu trừ:

$$\Delta PWM_\phi = K_{p_\phi} e_\phi + K_{i_\phi} I_\phi - K_{d_\phi} p \quad (154)$$

$$\Delta PWM_\theta = K_{p_\theta} e_\theta + K_{i_\theta} I_\theta - K_{d_\theta} q \quad (155)$$

Khi drone đang quay nhanh theo một chiều, thành phần $-K_{d\phi} p$ hoặc $-K_{d\theta} q$ tạo lực hãm để giảm dao động.

5.3 Tích phân và giới hạn output

Tích phân roll/pitch:

$$I_\phi(k) = I_\phi(k-1) + e_\phi(k) \Delta t \quad (156)$$

$$I_\theta(k) = I_\theta(k-1) + e_\theta(k) \Delta t \quad (157)$$

Trong code:

$$-1.0 \leq I_\phi, I_\theta \leq 1.0 \quad (158)$$

Output PWM correction cũng được kẹp:

$$-260 \leq \Delta PWM_\phi, \Delta PWM_\theta \leq 260 \quad (159)$$

Các giới hạn này làm cho attitude controller không thể yêu cầu mixer tạo ra lệnh quá mạnh so với khả năng motor.

6 Yaw Hold PID

Yaw controller giữ hướng ψ của drone. Trong project, yaw controller có thể bật/tắt bằng biến `enabled`.

6.1 Sai số yaw

Yaw target:

$$\psi_d \quad (160)$$

Yaw hiện tại:

$$\psi \quad (161)$$

Sai số yaw:

$$e_\psi = \text{wrap}(\psi_d - \psi) \quad (162)$$

Việc wrap góc đặc biệt quan trọng với yaw vì yaw có thể đi qua biên $-\pi$ và π .

6.2 Công thức yaw correction

Gọi yaw rate là:

$$r = \dot{\psi} \quad (163)$$

Tích phân yaw:

$$I_\psi(k) = I_\psi(k-1) + e_\psi(k)\Delta t \quad (164)$$

Output của yaw PID là PWM correction:

$$\Delta PWM_\psi = K_{p_\psi} e_\psi + K_{i_\psi} I_\psi - K_{d_\psi} r \quad (165)$$

Trong code, output này không phải torque trực tiếp. Nó là lượng PWM sẽ được mixer cộng/trừ lên các motor để tạo chênh lệch torque phản lực.

Giới hạn tích phân:

$$-0.8 \leq I_\psi \leq 0.8 \quad (166)$$

Giới hạn output:

$$-120 \leq \Delta PWM_\psi \leq 120 \quad (167)$$

6.3 Vai trò của yaw correction trong quadcopter

Roll và pitch được tạo chủ yếu bằng chênh lệch lực nâng giữa các motor. Yaw được tạo bằng chênh lệch torque phản lực của các cánh quạt quay ngược chiều nhau. Vì vậy mixer sẽ cộng ΔPWM_ψ vào nhóm motor này và trừ khỏi nhóm motor kia.

Nếu yaw hold tắt, project đặt:

$$\Delta PWM_\psi = 0 \quad (168)$$

Khi đó mixer chỉ còn giữ độ cao, roll và pitch.

7 Quad X PWM Mixer

Mixer là tầng nhận các lệnh PWM trừu tượng và tạo PWM cụ thể cho từng motor. Cấu hình của drone là quad X với thứ tự motor:

Motor	Vị trí trên frame	Tọa độ tương đối
1	front-left	$(+arm, +arm, z_m)$
2	front-right	$(+arm, -arm, z_m)$
3	rear-right	$(-arm, -arm, z_m)$
4	rear-left	$(-arm, +arm, z_m)$

Trong code:

$$arm = 0.159, \quad z_m = 0.04 \quad (169)$$

7.1 Công thức mixer

Mixer nhận:

$$PWM_{base}, \quad \Delta PWM_{\phi}, \quad \Delta PWM_{\theta}, \quad \Delta PWM_{\psi} \quad (170)$$

và tính:

$$PWM_1 = PWM_{base} + \Delta PWM_{\phi} - \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (171)$$

$$PWM_2 = PWM_{base} - \Delta PWM_{\phi} - \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (172)$$

$$PWM_3 = PWM_{base} - \Delta PWM_{\phi} + \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (173)$$

$$PWM_4 = PWM_{base} + \Delta PWM_{\phi} + \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (174)$$

Sau đó từng PWM được giới hạn:

$$1100 \leq PWM_i \leq 1940 \quad (175)$$

7.2 Ý nghĩa dấu trong mixer

Theo comment trong code, lực đẩy theo trục body-z tạo moment roll xấp xỉ theo yF và moment pitch xấp xỉ theo $-xF$. Vì vậy:

- Roll correction tăng motor phía trái và giảm motor phía phải.
- Pitch correction tăng motor phía sau và giảm motor phía trước.
- Yaw correction tăng nhóm motor có chiều quay này và giảm nhóm motor có chiều quay ngược lại.

Dấu của mixer phải khớp với thứ tự motor, vị trí motor và chiều quay cánh quạt. Nếu đổi thứ tự motor hoặc đổi chiều quay motor, công thức mixer cũng phải được kiểm tra lại.

8 Motor Model

Motor model mô phỏng đơn giản ESC, motor BLDC và propeller. Input là PWM, output là lực đẩy, dòng điện, RPM và torque phản lực.

8.1 Đổi PWM sang throttle

PWM được đưa về tỉ lệ ga u :

$$u = \frac{PWM - PWM_{min}}{PWM_{max} - PWM_{min}} \quad (176)$$

Với:

$$0 \leq u \leq 1 \quad (177)$$

Điện áp hiệu dụng đưa vào motor:

$$V_m = uV_{bat} \quad (178)$$

Trong code:

$$V_{bat} = 14.8 \text{ V} \quad (179)$$

8.2 Dòng điện và torque motor

Hằng số back-EMF được tính từ KV:

$$K_e = \frac{60}{2\pi KV} \quad (180)$$

Code dùng:

$$K_t = K_e \quad (181)$$

Dòng điện motor:

$$I = \frac{V_m - K_e\omega}{R_m} \quad (182)$$

Dòng điện được giới hạn:

$$0 \leq I \leq I_{max} \quad (183)$$

Torque motor:

$$\tau_m = K_t \max(I - I_0, 0) \quad (184)$$

Trong đó I_0 là dòng không tải.

8.3 Động lực học rotor

Torque tải khí động của propeller:

$$\tau_{load} = k_q\omega^2 \quad (185)$$

Gia tốc góc rotor:

$$\dot{\omega} = \frac{\tau_m - \tau_{load}}{J} \quad (186)$$

Cập nhật rời rạc:

$$\omega(k) = \omega(k-1) + \dot{\omega}\Delta t \quad (187)$$

Sau đó ω được kẹp trong khoảng:

$$0 \leq \omega \leq \omega_{max} \quad (188)$$

Trong code:

$$\omega_{max} = 1800 \text{ rad/s} \quad (189)$$

8.4 Lực đẩy, torque phản lực và RPM

Lực đẩy propeller:

$$T = k_f \omega^2 \quad (190)$$

Torque phản lực:

$$Q = k_q \omega^2 \quad (191)$$

RPM:

$$RPM = \frac{\omega 60}{2\pi} \quad (192)$$

Các tham số mặc định của motor model:

Tham số	Giá trị
KV	1100
R_m	0.073Ω
I_0	0.9 A
I_{max}	30 A
J	2.5×10^{-5}
k_f	4.0×10^{-6}
k_q	1.0×10^{-7}

9 Apply lực vào Isaac Sim

Sau khi motor model trả về lực đẩy T_i , controller apply lực lên rigid body `base_link` tại vị trí của từng motor:

$$\mathbf{F}_i = \begin{bmatrix} 0 \\ 0 \\ T_i \end{bmatrix} \quad (193)$$

Vì lực được đặt lệch khỏi tâm drone, Isaac Sim tự sinh moment roll/pitch tương ứng. Đây là lý do mixer không cần apply moment roll/pitch trực tiếp; nó chỉ cần tạo chênh lệch lực motor.

9.1 Torque phản lực yaw

Mỗi propeller tạo torque phản lực Q_i . Với chiều quay motor:

$$d = [1, -1, 1, -1] \quad (194)$$

Torque yaw tổng lên body được tính:

$$M_z = \sum_{i=1}^4 -d_i Q_i \quad (195)$$

Trong code, torque yaw này được đổi thành một cặp lực ngang đặt tại hai điểm đối xứng:

$$F_y = \frac{M_z}{2arm} \quad (196)$$

Một lực $+F_y$ đặt tại $x = +arm$, lực còn lại $-F_y$ đặt tại $x = -arm$. Hai lực này triệt tiêu tổng lực tuyến tính nhưng tạo moment quanh trục z .

9.2 Kết nối giữa motor model và simulator

Mối quan hệ cuối cùng giữa PWM và chuyển động drone là:

$$PWM_i \longrightarrow \omega_i \longrightarrow T_i, Q_i \longrightarrow IsaacSim \longrightarrow x, y, z, \phi, \theta, \psi \quad (197)$$

Như vậy, controller không trực tiếp thay đổi pose của drone. Controller chỉ apply lực vật lý, còn chuyển động thật do engine physics của Isaac Sim tích phân.

10 Propeller Spinner

File `propeller_spinner.py` không phải là PID controller, nhưng giúp cánh quạt trong Isaac Sim quay theo RPM của motor model. Module này dùng `dynamic_control` để tìm các DOF:

$$propeller_1_joint, propeller_2_joint, propeller_3_joint, propeller_4_joint \quad (198)$$

Nếu tìm được DOF, code cấu hình velocity drive và đặt target velocity cho từng cánh quạt.

10.1 Đổi RPM sang vận tốc joint

Với motor thứ i , RPM từ motor model được đổi sang rad/s:

$$\omega_i = RPM_i \frac{2\pi}{60} \quad (199)$$

Lệnh vận tốc joint:

$$\omega_{cmd,i} = d_i s \omega_i \quad (200)$$

Trong đó:

- d_i là chiều quay motor, mặc định $[1, -1, 1, -1]$.
- s là hệ số hiển thị, mặc định 0.7.

Việc nhân với s giúp tốc độ quay hiển thị không quá cực đoan so với mô phỏng. Phần này chủ yếu phục vụ trực quan, không phải nguồn lực chính tác động lên drone. Lực chính vẫn đến từ `MotorModel` và `apply_body_force`.

11 Tracking Logger

File `tracking_logger.py` ghi log trạng thái drone và target ra CSV. Logger được bật bằng:

$$start_tracking_log(csv_path, sample_period) \quad (201)$$

Mặc định:

$$sample_period = 0.02 \text{ s} \quad (202)$$

Tức là tần số ghi log tối đa khoảng:

$$50 \text{ Hz} \quad (203)$$

11.1 Các đại lượng được ghi

CSV log gồm các nhóm dữ liệu:

- Thời gian mô phỏng: `time`.
- Vị trí và target: x, x_d, y, y_d, z, z_d .
- Roll, pitch, yaw theo độ.

- Roll, pitch, yaw target theo độ.
- Roll, pitch, yaw theo radian.
- Roll, pitch, yaw target theo radian.

Các dữ liệu này đủ để vẽ đồ thị tracking, kiểm tra overshoot, kiểm tra sai số xác lập và so sánh target với trạng thái thật.

12 Compatibility Mixin

File `attitude_hold_compat.py` chứa `AttitudeHoldCompatibilityMixin`. Module này không tạo luật điều khiển mới. Vai trò của nó là giữ lại API cũ sau khi code được tách thành nhiều controller nhỏ.

Ví dụ, khi code bên ngoài truy cập:

$$kp_x \quad (204)$$

thì property trong mixin thực chất đọc:

$$position_controller.kp_x \quad (205)$$

Tương tự:

$$z_target \leftrightarrow altitude_controller.z_target \quad (206)$$

$$yaw_target \leftrightarrow yaw_controller.yaw_target \quad (207)$$

$$roll_target, pitch_target \leftrightarrow attitude_controller \quad (208)$$

Cách làm này giúp những script cũ vẫn chạy được, đồng thời code mới vẫn có cấu trúc rõ ràng hơn.

13 Tóm tắt toàn bộ hệ thống

Toàn bộ controller có thể tóm tắt bằng chuỗi biến đổi:

$$(x_d, y_d, z_d, \psi_d) - (x, y, z, \psi) \longrightarrow PID \longrightarrow PWM_i \longrightarrow T_i, Q_i \longrightarrow \text{Isaac Sim physics} \quad (209)$$

Trong đó:

- Position PID biến sai số x, y thành góc nghiêng mong muốn.
- Altitude PID biến sai số z thành PWM nền.
- Attitude PID biến sai số roll/pitch thành PWM correction.
- Yaw PID biến sai số yaw thành PWM correction.
- Mixer tạo PWM cho từng motor theo cấu hình quad X.
- Motor model biến PWM thành lực đẩy và torque phản lực.

- Isaac Sim nhận lực vật lý và cập nhật trạng thái drone.

Điểm quan trọng nhất là mỗi PID chỉ tạo output mà tầng sau cần. Nhờ vậy hệ điều khiển dễ đọc, dễ chỉnh tham số và dễ debug: khi drone lỗi độ cao thì kiểm tra altitude PID; khi nghiêng sai thì kiểm tra attitude PID/mixer; khi trôi vị trí ngang thì kiểm tra position PID và quy ước dấu roll/pitch.

References

- [1] K. J. Astrom and T. Hagglund, *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America, 1995.
- [2] S. Bouabdallah, “Design and control of quadrotors with application to autonomous flying,” Ph.D. dissertation, Ecole Polytechnique Federale de Lausanne, 2007.
- [3] R. Mahony, V. Kumar, and P. Corke, “Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 3, pp. 20–32, 2012.